

Natural Language Processing 2

FFNN for NLP Classification

Fatemeh Dehrouyeh



FANSHAWE

- 1 Vectorization vs. Word Embedding in FFNN
 - Vectorization in FFNN
 - Word Embedding in FFNN

Current Section

1 Vectorization vs. Word Embedding in FFNN

- Vectorization in FFNN
- Word Embedding in FFNN

How FFNN is used in NLP:

- Text data must first be converted into numerical vectors using methods such as TF-IDF or word embeddings.
- These numerical vectors are passed into a Feed-Forward Neural Network.
- The FFNN learns patterns in the text representation and predicts the class label.

Fake News Classification Dataset

Consider the "Fake News Classification" dataset from Kaggle (**Source:** [Link](#)):

▲ title Title	▲ text Text	# label Label
62348 unique values	[empty] 1% Killing Obama adm... 0% Other (71338) 99%	 0
LAW ENFORCEMENT ON HIGH ALERT Following Threats Against Cops And Whites On 9-11By #BlackLivesMatter ...	No comment is expected from Barack Obama Members of the #FYF911 or #FukYoFlag and #BlackLivesMatter ...	1
	Did they post their votes for Hillary already?	1

This dataset is designed for binary text classification in Natural Language Processing (NLP). The goal is to predict whether a news article is fake or real based on textual features such as the article title and body text.

Current Subsection

1 Vectorization vs. Word Embedding in FFNN

- Vectorization in FFNN
- Word Embedding in FFNN

How TF-IDF Works with FFNN in NLP

Step 1: Build a vocabulary

- The dataset is scanned to collect important words from all documents.
- Example: if the vocabulary has 10,000 words, then each document will be represented by a vector of length 10,000.

Step 2: Convert each document into a TF-IDF vector

- Each document becomes a numerical vector.
- Each value measures how important a word is in that document compared to the whole dataset.

Document $\rightarrow [x_1, x_2, x_3, \dots, x_{10000}]$

Step 3: Feed the vector into the FFNN

- The FFNN input layer has the same size as the TF-IDF vector.
- If the TF-IDF vector has 10,000 features, the input layer receives 10,000 input values.
- The network trains weights that connect these input features to hidden neurons.

TF-IDF Vector $\rightarrow$ FFNN Hidden Layers $\rightarrow$ Fake or Real

Key Idea: TF-IDF turns text into numbers. The FFNN then learns which word patterns are useful for classification.

Code Example—Vectorization in FFNN

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping

# 1. Load dataset
df = pd.read_csv("WELFake_Dataset.csv", engine="python", on_bad_lines="skip")

# 2. Basic cleaning
df = df.dropna(subset=["title", "text", "label"])
df["label"] = pd.to_numeric(df["label"], errors="coerce")
df = df.dropna(subset=["label"])
df["label"] = df["label"].astype(int)
df = df[df["label"].isin([0, 1])]
df["content"] = df["title"].astype(str) + " " + df["text"].astype(str)

# 3. Split features and target
X = df["content"]
y = df["label"]

# 4. Train, validation, test split
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.30, random_state=42, stratify=y)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.50, random_state=42, stratify=y_temp)

# 5. Convert text into TF-IDF vectors
tfidf = TfidfVectorizer(max_features=10000, stop_words="english")
X_train_tfidf = tfidf.fit_transform(X_train)
X_val_tfidf = tfidf.transform(X_val)
X_test_tfidf = tfidf.transform(X_test)
```

Code Example—Vectorization in FFNN—Continued

```
# 6. Build FFNN model
model = Sequential([
    Dense(128, activation="relu", input_shape=(X_train_tfidf.shape[1],)),
    Dropout(0.3),
    Dense(64, activation="relu"),
    Dropout(0.3),
    Dense(1, activation="sigmoid")
])

# 7. Compile model
model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])

# 8. Early stopping
early_stop = EarlyStopping(monitor="val_loss", patience=3, restore_best_weights=True)

# 9. Train model
history = model.fit(X_train_tfidf, y_train, validation_data=(X_val_tfidf, y_val),
                    epochs=10, batch_size=64, callbacks=[early_stop], verbose=1)

# 10. Evaluate model
test_loss, test_accuracy = model.evaluate(X_test_tfidf, y_test, verbose=0)
print("Test Loss:", test_loss)
print("Test Accuracy:", test_accuracy)

# 11. Predictions
y_pred_prob = model.predict(X_test_tfidf)
y_pred = (y_pred_prob >= 0.5).astype(int).flatten()

print("Accuracy Score:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=["Fake", "Real"]))

# 12. Example predictions
results = pd.DataFrame({"Actual Label": y_test.values, "Predicted Label": y_pred, "Real Probability": y_pred_prob.flatten()})
print(results.head(10))
```

Code Example—Vectorization—Result

```
Epoch 1/10
783/783 ————— 20s 24ms/step - accuracy: 0.9250 - loss: 0.1970 - val_accuracy: 0.9524 - val_loss: 0.1273
Epoch 2/10
783/783 ————— 19s 22ms/step - accuracy: 0.9722 - loss: 0.0795 - val_accuracy: 0.9534 - val_loss: 0.1252
Epoch 3/10
783/783 ————— 20s 26ms/step - accuracy: 0.9880 - loss: 0.0383 - val_accuracy: 0.9561 - val_loss: 0.1441
Epoch 4/10
783/783 ————— 17s 22ms/step - accuracy: 0.9951 - loss: 0.0169 - val_accuracy: 0.9549 - val_loss: 0.1770
Epoch 5/10
783/783 ————— 17s 22ms/step - accuracy: 0.9972 - loss: 0.0098 - val_accuracy: 0.9526 - val_loss: 0.2095
Test Loss: 0.13206177949905396
Test Accuracy: 0.952846884727478
336/336 ————— 2s 7ms/step
Accuracy Score: 0.9528468921815302
```

Confusion Matrix:

```
[[4970  285]
 [ 221 5255]]
```

Classification Report:

	precision	recall	f1-score	support
Fake	0.96	0.95	0.95	5255
Real	0.95	0.96	0.95	5476
accuracy			0.95	10731
macro avg	0.95	0.95	0.95	10731
weighted avg	0.95	0.95	0.95	10731

	Actual Label	Predicted Label	Real Probability
0	1	1	0.947898
1	1	1	0.999950
2	0	0	0.017165
3	1	1	0.965911
4	1	1	0.676980
5	0	0	0.002014
6	0	1	0.709123
7	0	0	0.000008
8	0	0	0.002944
9	0	0	0.000390

Current Subsection

1 Vectorization vs. Word Embedding in FFNN

- Vectorization in FFNN
- Word Embedding in FFNN

Popular Word Embedding Methods

Word embeddings convert words into dense numerical vectors, where words with similar meanings have similar vector representations.

● Word2Vec

- Learns word vectors from surrounding context words.
- Popular models: Continuous Bag-of-Words (CBOW) and Skip-Gram.
- Example idea: words used in similar contexts get similar vectors.

● GloVe

- Learns embeddings using global word co-occurrence statistics.
- Focuses on how often words appear together in a large corpus.
- Useful for capturing broad semantic relationships between words.

● FastText

- Represents each word using smaller character-level parts.
- Handles rare words and misspelled words better than Word2Vec.
- Useful when words share prefixes, suffixes, or roots.

Key Idea: Word embeddings allow neural networks to process text by turning words into meaningful numerical vectors.

Popular Word Embeddings

```
!pip install gensim
import gensim.downloader as api
import numpy as np

# 1. Choose a word
word = "computer"

# 2. Load pretrained embedding models
word2vec_model = api.load("word2vec-google-news-300")
glove_model = api.load("glove-wiki-gigaword-50")
fasttext_model = api.load("fasttext-wiki-news-subwords-300")

# 3. Get vectors
word2vec_vector = word2vec_model[word]
glove_vector = glove_model[word]
fasttext_vector = fasttext_model[word]

# 4. Print vector sizes
print("Word2Vec vector size:", word2vec_vector.shape)
print("GloVe vector size:", glove_vector.shape)
print("FastText vector size:", fasttext_vector.shape)

# 5. Print first 10 values from each vector
print("\nWord2Vec vector for:", word)
print(word2vec_vector[:10])

print("\nGloVe vector for:", word)
print(glove_vector[:10])

print("\nFastText vector for:", word)
print(fasttext_vector[:10])
```

Popular Word Embeddings—Result

```
Word2Vec vector size: (300,)
```

```
GloVe vector size: (50,)
```

```
FastText vector size: (300,)
```

```
Word2Vec vector for: computer
```

```
[ 0.10742188 -0.20117188  0.12304688  0.21191406 -0.09130859  0.21679688  
 -0.13183594  0.08300781  0.20214844  0.04785156]
```

```
GloVe vector for: computer
```

```
[ 0.079084 -0.81504  1.7901  0.91653  0.10797 -0.55628 -0.84427  
 -1.4951  0.13418  0.63627 ]
```

```
FastText vector for: computer
```

```
[-0.089791 -0.017821 -0.037268 -0.013295 -0.0090625 -0.03825  
 -0.026104 -0.089276  0.051467 -0.0021857]
```

How Word Embeddings Work with FFNN in NLP

Step 1: Convert words into embedding vectors

- Each word is represented by a dense vector of numbers.

$$\text{word} \rightarrow [e_1, e_2, e_3, \dots, e_d]$$

Step 2: Convert a document into many word vectors

- A document is first split into words or tokens.
- Each token is replaced by its embedding vector.
- This creates a matrix:

$$\text{Document} \rightarrow \begin{bmatrix} \text{word}_1 \\ \text{word}_2 \\ \vdots \\ \text{word}_n \end{bmatrix} \rightarrow \begin{bmatrix} e_{1,1} & e_{1,2} & \cdots & e_{1,d} \\ e_{2,1} & e_{2,2} & \cdots & e_{2,d} \\ \vdots & \vdots & \ddots & \vdots \\ e_{n,1} & e_{n,2} & \cdots & e_{n,d} \end{bmatrix}$$

- Different documents may have different numbers of tokens, so n can change from one document to another.

Step 3: Combine word vectors for an FFNN

- A basic FFNN needs one fixed-size input vector. Since documents may have different lengths, we combine the word vectors into a fixed-size representation.
- Common methods include averaging, summing, or padding/truncating then flattening.

$$\text{Word Embeddings} \rightarrow \text{Average/Sum/Flatten} \rightarrow \text{FFNN} \rightarrow \text{Class}$$

Code Example—Word Embedding in FFNN

```
import pandas as pd
import numpy as np
import re
import gensim.downloader as api
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping

# 1. Load dataset
df = pd.read_csv("WELFake_Dataset.csv", engine="python", on_bad_lines="skip")

# 2. Basic cleaning
df = df.dropna(subset=["title", "text", "label"])
df["label"] = pd.to_numeric(df["label"], errors="coerce")
df = df.dropna(subset=["label"])
df["label"] = df["label"].astype(int)
df = df[df["label"].isin([0, 1])]
df["content"] = df["title"].astype(str) + " " + df["text"].astype(str)

# 3. Split features and target
X = df["content"]
y = df["label"]

# 4. Train, validation, test split
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.30, random_state=42, stratify=y)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.50, random_state=42, stratify=y_temp)
```

Code Example—Word Embedding in FFNN—Continued

```
# 5. Choose and load word embedding model
embedding_choice = "glove"
# embedding_choice = "word2vec"
# embedding_choice = "fasttext"

if embedding_choice == "glove":
    embedding_model = api.load("glove-wiki-gigaword-100")
elif embedding_choice == "word2vec":
    embedding_model = api.load("word2vec-google-news-300")
elif embedding_choice == "fasttext":
    embedding_model = api.load("fasttext-wiki-news-subwords-300")

embedding_dim = embedding_model.vector_size
print("Embedding model:", embedding_choice)
print("Embedding dimension:", embedding_dim)

# 6. Tokenizer
def tokenize(text):
    text = text.lower()
    text = re.sub(r"^[^a-zA-Z]", " ", text)
    return text.split()

# 7. Convert each document into one average embedding vector
def document_to_vector(text):
    words = tokenize(text)
    vectors = [embedding_model[word] for word in words if word in embedding_model]
    if len(vectors) == 0:
        return np.zeros(embedding_dim)
    return np.mean(vectors, axis=0)

X_train_embed = np.array([document_to_vector(text) for text in X_train])
X_val_embed = np.array([document_to_vector(text) for text in X_val])
X_test_embed = np.array([document_to_vector(text) for text in X_test])
```

Code Example—Word Embedding in FFNN—Continued

```
# 8. Build FFNN model
model = Sequential([
    Dense(128, activation="relu", input_shape=(X_train_embed.shape[1],)),
    Dropout(0.3),
    Dense(64, activation="relu"),
    Dropout(0.3),
    Dense(1, activation="sigmoid")
])

# 9. Compile model
model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])

# 10. Early stopping
early_stop = EarlyStopping(monitor="val_loss", patience=3, restore_best_weights=True)

# 11. Train model
history = model.fit(X_train_embed, y_train, validation_data=(X_val_embed, y_val), epochs=10, batch_size=64, callbacks=[early_stop], verbose=1)

# 12. Evaluate model
test_loss, test_accuracy = model.evaluate(X_test_embed, y_test, verbose=0)
print("Test Loss:", test_loss)
print("Test Accuracy:", test_accuracy)

# 13. Predictions
y_pred_prob = model.predict(X_test_embed)
y_pred = (y_pred_prob >= 0.5).astype(int).flatten()

print("Accuracy Score:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=["Fake", "Real"]))

# 14. Example predictions
results = pd.DataFrame({"Actual Label": y_test.values, "Predicted Label": y_pred, "Real Probability": y_pred_prob.flatten()})
print(results.head(10))
```

Code Example—Word Embedding in FFNN—Result

```
Epoch 7/10
783/783 ————— 2s 3ms/step - accuracy: 0.8654 - loss: 0.3195 - val_accuracy: 0.8666 - val_loss: 0.3133
Epoch 8/10
783/783 ————— 2s 3ms/step - accuracy: 0.8656 - loss: 0.3173 - val_accuracy: 0.8723 - val_loss: 0.3068
Epoch 9/10
783/783 ————— 2s 3ms/step - accuracy: 0.8687 - loss: 0.3116 - val_accuracy: 0.8686 - val_loss: 0.3064
Epoch 10/10
783/783 ————— 3s 3ms/step - accuracy: 0.8697 - loss: 0.3099 - val_accuracy: 0.8669 - val_loss: 0.3142
Test Loss: 0.3080041706562042
Test Accuracy: 0.8678594827651978
336/336 ————— 1s 1ms/step
Accuracy Score: 0.8678594725561457
```

Confusion Matrix:

```
[[4672  583]
 [ 835 4641]]
```

Classification Report:

	precision	recall	f1-score	support
Fake	0.85	0.89	0.87	5255
Real	0.89	0.85	0.87	5476
accuracy			0.87	10731
macro avg	0.87	0.87	0.87	10731
weighted avg	0.87	0.87	0.87	10731

	Actual Label	Predicted Label	Real Probability
0	1	0	0.383603
1	1	1	0.933911
2	0	0	0.052273
3	1	1	0.864830
4	1	1	0.973097
5	0	0	0.023385
6	0	1	0.820825
7	0	0	0.011706
8	0	0	0.002655
9	0	0	0.034942

Congratulations!

Now you know how feed forward neural networks work in NLP!

